

Day 9 — Boolean Expressions and Universal Gates

Target time: 90–120 minutes

Suggested rhythm: 25 minutes deriving the gates · 25 minutes building and checking truth tables · 25 minutes on De Morgan and active-low logic · 20 minutes on universal gates · 10–25 minutes for the hands-on exercise and checkpoint

Two circuits can look completely different and still produce the same output for every possible input. How can we prove that—and why might an engineer still prefer one circuit over the other?

Why this lesson exists

Day 8 established the physical intuition for NOT, AND, and OR: inversion, series requirements, parallel alternatives, and the historical path from switching networks to electronic gates. Today we treat those gates as pieces of a design language.

A Boolean expression is a compact recipe for a circuit. It says which operations feed which later operations. A truth table is the recipe's complete behavioral contract: for every allowed input combination, it records the output.

That gives us a disciplined path:

```
requirement
  → Boolean expression
  → truth table
  → gate network
  → physical implementation
```

The same path also runs backward. Given a circuit, we can recover an expression and test what it does.

By the end, you should be able to:

- derive XOR, NAND, and NOR from NOT, AND, and OR;
- compare all six gates without relying on a memorized picture;
- translate a Boolean expression into a gate-by-gate circuit recipe;
- construct a truth table systematically, including intermediate columns;
- derive and verify both of De Morgan's laws;
- read an active-low signal without treating 0 as universally “off”;
- build NOT, AND, and OR using only NAND gates or only NOR gates;
- explain why equivalent circuits can differ in speed, area, power, fan-out, and implementation convenience.

We will connect these ideas to CPU logic, but we will not build an adder today. Day 10 will give arithmetic and larger combinational circuits the room they deserve.

1. Three familiar gates, then three derived ones

Use these notations:

NOT A	$\neg A$
A AND B	$A \wedge B$
A OR B	$A \vee B$
A XOR B	$A \oplus B$

In code, symbols differ. Python uses `not`, `and`, and `or` for logical operations and `^` for bitwise XOR. C has logical `!`, `&&`, and `||`, plus bitwise `~`, `&`, `|`, and `^`. Today our variables are restricted to the logical values `0` and `1`; do not assume that logical and bitwise operators are interchangeable for arbitrary integers.

NOT, AND, and OR as our starting vocabulary

NOT: reverse one input
AND: all stated conditions must be 1
OR: at least one stated condition must be 1

OR is inclusive. When $A = 1$ and $B = 1$, $A \vee B = 1$. Everyday speech sometimes uses “or” to mean “one or the other, but not both.” Boolean OR does not.

Deriving XOR: accept either unequal case

Suppose we want the output to be 1 when the two inputs differ.

There are exactly two acceptable cases:

A is 1 AND B is 0
OR
A is 0 AND B is 1

Write each case, then join them:

$A \wedge \neg B$
 $\neg A \wedge B$
 $A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$

That is XOR—exclusive OR. It accepts exactly one asserted input for two-input XOR. “The inputs differ” is often the clearest mental model.

Notice that we did not invent XOR by memorizing a symbol. We described the rows we wanted and assembled an expression that selects those rows.

Deriving NAND: reject only the all-1 case

Start with AND:

$A \wedge B$

Invert its result:

$$\text{NAND}(A, B) = \neg(A \wedge B)$$

AND is 1 only at $A = 1, B = 1$, so NAND is 0 only there. NAND means “not both.”

Deriving NOR: accept only the all-0 case

Start with OR and invert:

$$\text{NOR}(A, B) = \neg(A \vee B)$$

OR is 0 only at $A = 0, B = 0$, so NOR is 1 only there. NOR means “neither.”

Predict before reading the table

For each input pair, answer from the verbal rule:

1. If both inputs are 1, what do XOR, NAND, and NOR output?
2. If both inputs are 0, what do they output?
3. If the inputs differ, what do they output?
4. Which derived gate differs from AND in every row?
5. Which derived gate differs from OR in every row?

Now compare all six two-input gates:

A	B	AND	OR	XOR	NAND	NOR
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	1	0
1	1	1	1	0	0	0

NOT is unary, so it has its own two-row table:

A	NOT A
0	1
1	0

Read the table by patterns, not as 26 isolated cells:

- AND has one 1, at the all-1 row.
- OR has one 0, at the all-0 row.
- XOR has 1 on the two unequal rows.
- NAND is the row-by-row complement of AND.
- NOR is the row-by-row complement of OR.
- NOT complements its one input.

These patterns are enough to reconstruct the tables if memory fails.

2. An expression is a circuit recipe

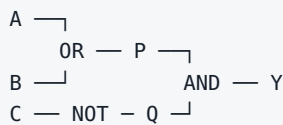
Consider:

$$Y = (A \vee B) \wedge \neg C$$

Parentheses and operator order describe dependencies. One direct circuit recipe is:

1. Feed A and B into OR. Call the result P.
2. Feed C into NOT. Call the result Q.
3. Feed P and Q into AND. Call the result Y.

Diagrammed as data flow:



The temporary names **P** and **Q** are not extra mathematical meaning. They name wires carrying intermediate results.

Structure matters

These expressions are different recipes:

$$\begin{aligned} Y_1 &= (A \vee B) \wedge C \\ Y_2 &= A \vee (B \wedge C) \end{aligned}$$

In the first, OR happens before AND. In the second, AND happens before OR. Try

$A = 1, B = 0, C = 0$:

$$\begin{aligned} Y_1 &= (1 \vee 0) \wedge 0 = 1 \wedge 0 = 0 \\ Y_2 &= 1 \vee (0 \wedge 0) = 1 \vee 0 = 1 \end{aligned}$$

One counterexample proves the expressions are not equivalent.

Do not rely on ordinary-language rhythm to parse an expression. Use explicit parentheses while learning and while communicating a design.

Expressions describe settled behavior

The recipe is an abstraction. A real gate has propagation delay. When an input changes, intermediate wires may change at different times before the whole circuit settles. Today's truth tables describe stable input and output values after propagation.

This distinction will matter later:

Boolean expression: desired steady-state mapping
physical circuit: devices, voltages, delays, loading, and transitions

Both descriptions are valid, but they answer different questions.

3. Build truth tables systematically

For n independent binary inputs, there are 2^n input combinations:

```
1 input  → 2 rows
2 inputs → 4 rows
3 inputs → 8 rows
4 inputs → 16 rows
```

The safest row order is binary counting. For three inputs:

```
A B C
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1
```

C changes every row, **B** every two rows, and **A** every four. This prevents missing or duplicating a combination.

Use one column per intermediate result

Derive the table for:

```
Y = (A ⊕ B) ∧ ¬C
```

Do not try to evaluate the full expression in your head. Name its parts:

```
P = A ⊕ B
Q = ¬C
Y = P ∧ Q
```

Then work left to right:

A	B	C	P = A XOR B	Q = NOT C	Y = P AND Q
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	0	1	0
1	1	1	0	0	0

The result is 1 only when **A** and **B** differ and **C** is 0.

A repeatable truth-table method

1. Count the inputs and create exactly 2^n rows.
2. Fill input columns by binary counting.
3. Parenthesize the expression.
4. Create one column for each intermediate subexpression.
5. Evaluate each column from already completed columns.
6. Translate the final 1 rows back into words.
7. Spot-check a few rows directly from the original requirement.

The final two steps catch errors that neat-looking arithmetic can hide.

Prediction pause

Without constructing all eight rows, predict:

$$F = \neg(A \vee B) \vee C$$

1. If $C = 1$, can F ever be 0?
2. If $C = 0$, what must A and B be for $F = 1$?
3. How many of the eight rows should produce 1?

Write the predictions down. You will test them in the hands-on section.

4. De Morgan's laws: move inversion through a boundary

De Morgan's laws are often presented as formulas to memorize:

$$\begin{aligned}\neg(A \wedge B) &= \neg A \vee \neg B \\ \neg(A \vee B) &= \neg A \wedge \neg B\end{aligned}$$

It is better to derive their meaning.

Derive the first law in words

Start with:

$$\text{NOT } (A \text{ AND } B)$$

$A \text{ AND } B$ is true only when both are true. Its negation therefore says:

it is not the case that both are true

For that to hold, at least one input must be false:

$$(\text{NOT } A) \text{ OR } (\text{NOT } B)$$

So:

$$\neg(A \wedge B) = \neg A \vee \neg B$$

This is also the NAND function.

Verify the first law completely

A	B	A AND B	NOT(A AND B)	NOT A	NOT B	(NOT A) OR (NOT B)
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

The two final-function columns match in every row. That is exhaustive evidence for two Boolean inputs.

Derive the second law in words

Start with:

NOT (A OR B)

A OR B is true when either input is true. To negate that statement, neither input may be true:

(NOT A) AND (NOT B)

So:

$\neg(A \vee B) = \neg A \wedge \neg B$

This is the NOR function.

Verify the second law completely

A	B	A OR B	NOT(A OR B)	NOT A	NOT B	(NOT A) AND (NOT B)
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Again, the compared columns match on every row.

The transformation rule

When an inversion crosses an AND/OR boundary:

AND changes to OR
OR changes to AND
every input at the boundary is inverted

You must do both parts. Changing AND to OR without complementing the inputs is wrong. Complementing the inputs without changing the operator is also wrong.

For a larger example:

```
¬(A ∧ B ∧ C) = ¬A ∨ ¬B ∨ ¬C
¬(A ∨ B ∨ C) = ¬A ∧ ¬B ∧ ¬C
```

A common mistake

This is false:

```
¬(A ∧ B) = ¬A ∧ ¬B
```

Try $A = 0$, $B = 1$:

```
left  = ¬(0 ∧ 1) = ¬0 = 1
right = ¬0 ∧ ¬1 = 1 ∧ 0 = 0
```

A single mismatching row disproves equivalence.

5. Active-low logic: asserted does not always mean HIGH

We often casually say:

```
1 = active
0 = inactive
```

That is a convention, not a law of nature. An **active-low** signal is asserted when its electrical/logical level is LOW.

Common notation includes:

```
RESET_n
/RESET
RESET#
```

The spelling varies by project. The essential information is that the signal performs its named action when its level is 0.

For an active-low reset:

RESET_n	Meaning
0	reset is asserted; reset action requested
1	reset is deasserted; normal operation allowed

So the sentence “reset is 0” is ambiguous. It could mean the wire is LOW, or someone might incorrectly mean the reset action is inactive. Say “the RESET_n level is LOW” or “reset is asserted.”

De Morgan makes active-low networks readable

Imagine two active-low fault signals, FAULT_A_n and FAULT_B_n. Each falls to 0 when its fault occurs.

We want `OK_n` to remain 1 only when neither fault is asserted:

$$\text{OK}_n = \text{FAULT_A}_n \wedge \text{FAULT_B}_n$$

If either input drops LOW, the AND output drops LOW. Read electrically, a LOW propagates the fault condition. Read in terms of positive fault assertions, the behavior is OR-like: fault A or fault B is enough to report a fault.

This is sometimes called **bubble logic**: inversions at gate inputs or outputs change how AND/OR behavior is interpreted. De Morgan's laws explain it exactly; there is no separate active-low algebra.

Predict

An active-low chip-select input is named `CS_n`.

1. Which level selects the chip?
2. If three conditions must all permit selection, would you reason about their voltage levels or their asserted meanings first?
3. Why can a diagram look like an AND network at the voltage level but describe OR-like behavior among active-low conditions?

The safe habit is to state both the signal level and the asserted meaning.

6. Functional completeness: one gate type can be enough

A set of gate types is **functionally complete** if circuits made only from that set can implement every Boolean function.

NAND alone is functionally complete. NOR alone is also functionally complete. We can establish this constructively:

1. show how the universal gate makes NOT;
2. use that NOT to make AND or OR;
3. obtain the remaining basic operation through De Morgan;
4. because NOT, AND, and OR can express any Boolean truth table, the one gate can express any Boolean function.

This is a statement about expressive power, not a claim that every practical chip should be built from identical two-input NANDs or NORs.

Build NOT, AND, and OR from NAND

Define:

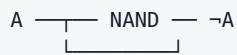
$$\text{NAND}(A, B) = \neg(A \wedge B)$$

NAND as NOT

Tie the two inputs together:

$$\text{NAND}(A, A) = \neg(A \wedge A) = \neg A$$

Because $A \wedge A = A$, one NAND becomes an inverter.



NAND as AND

First produce NAND, then invert it with a tied-input NAND:

$$\begin{aligned} P &= \text{NAND}(A, B) &= \neg(A \wedge B) \\ Y &= \text{NAND}(P, P) &= \neg P \\ & &= A \wedge B \end{aligned}$$

NAND as OR

De Morgan says:

$$A \vee B = \neg(\neg A \wedge \neg B)$$

Create both input inversions with NAND, then NAND those results:

$$\begin{aligned} P &= \text{NAND}(A, A) &= \neg A \\ Q &= \text{NAND}(B, B) &= \neg B \\ Y &= \text{NAND}(P, Q) &= \neg(\neg A \wedge \neg B) \\ & &= A \vee B \end{aligned}$$

Build NOT, OR, and AND from NOR

Define:

$$\text{NOR}(A, B) = \neg(A \vee B)$$

NOR as NOT

Tie the inputs together:

$$\text{NOR}(A, A) = \neg(A \vee A) = \neg A$$

NOR as OR

Produce NOR, then invert it:

$$\begin{aligned} P &= \text{NOR}(A, B) &= \neg(A \vee B) \\ Y &= \text{NOR}(P, P) &= \neg P \\ & &= A \vee B \end{aligned}$$

NOR as AND

De Morgan says:

$$A \wedge B = \neg(\neg A \vee \neg B)$$

Create both input inversions with NOR, then NOR those results:

$$\begin{aligned} P &= \text{NOR}(A, A) &&= \neg A \\ Q &= \text{NOR}(B, B) &&= \neg B \\ Y &= \text{NOR}(P, Q) &&= \neg(\neg A \vee \neg B) \\ &&&= A \wedge B \end{aligned}$$

Why NOT, AND, and OR can express any truth table

Take any row where a desired function outputs 1. Build an AND term that matches exactly that row.

For example, the three-input row $A = 1, B = 0, C = 1$ is selected by:

$$A \wedge \neg B \wedge C$$

That term is 1 only for 101. Build one such term for every output-1 row, then OR the terms together. This is a **sum-of-products** construction.

It may not be the smallest or fastest circuit, but it proves that NOT, AND, and OR are sufficient. Since NAND can build all three, NAND is sufficient. The same reasoning establishes NOR's completeness.

7. Same function, different circuit

These two expressions implement the same function:

$$\begin{aligned} Y_1 &= \neg(A \wedge B) \\ Y_2 &= \neg A \vee \neg B \end{aligned}$$

Their diagrams look different:

Y_1 : one NAND operation

Y_2 : invert A —┐
 OR — Y
 invert B —┐

Boolean equivalence means only this:

For every allowed stable input combination, the outputs match.

It does not mean the physical circuits are identical.

Why an engineer may care which equivalent form is used

Depending on the technology and design context, equivalent circuits can differ in:

- **gate count:** how many cells or devices are required;
- **logic depth:** how many gate stages lie on an input-to-output path;
- **propagation delay:** how long the output takes to settle;
- **area:** how much silicon or board space is consumed;
- **power and switching activity:** how much energy is used and how often internal nodes toggle;
- **fan-in:** how many inputs one gate accepts;
- **fan-out and loading:** how many later inputs a signal must drive;
- **signal polarity:** whether active-high or active-low interfaces fit naturally;
- **available cells:** which characterized gates exist in a chip library;
- **hazards:** whether unequal path delays can cause temporary glitches;
- **testability and routing:** how easily the implementation can be placed, connected, and tested.

“Fewer symbols on paper” is not automatically “better hardware.” A technology-mapping tool may replace a simple-looking expression with a larger network because that network meets timing, power, or layout constraints.

Likewise, functional completeness does not imply practical uniformity. NAND or NOR gates can implement everything, but real CPUs use libraries containing inverters, NANDs, NORs, complex gates, multiplexers, storage elements, and specialized structures.

The CPU connection

Inside a CPU, Boolean networks contribute to:

- deciding whether control conditions are satisfied;
- decoding instruction fields;
- selecting one data source from several;
- comparing bit patterns;
- masking or combining status and control signals;
- implementing arithmetic and bitwise datapaths.

At this layer, the circuit does not know that a pattern is an instruction, address, signed number, permission, or character. It settles according to its input voltages and topology. The surrounding architecture gives those bit patterns meaning.

Day 10 will combine gates into arithmetic and other combinational structures. For now, the important bridge is:

```
equivalent Boolean expressions
  → alternative gate networks
  → engineering choice among implementations
  → larger CPU logic blocks
```

8. Hands-on: predict, generate, and explain a truth table

Create a temporary Python file anywhere outside the lesson directory, or run the snippet directly with `python3`. This exercise does not require creating another course artifact.

Before running it, predict the answers to the questions below.

```
from itertools import product

def bit(value):
    return int(bool(value))

def evaluate(a, b, c):
    xor_derived = (a and not b) or (not a and b)
    nand = not (a and b)
    nor = not (a or b)

    f_original = not (a or b) or c
    f_rewritten = ((not a) and (not b)) or c

    nand_not_a = not (a and a)
    nand_not_b = not (b and b)
    nand_or = not (nand_not_a and nand_not_b)

    return tuple(map(bit, (
        xor_derived,
        nand,
        nor,
        f_original,
        f_rewritten,
        nand_or,
    )))

print("A B C | XOR NAND NOR | F original/rewrite | OR-via-NAND")
for a, b, c in product((0, 1), repeat=3):
    xor, nand, nor, f1, f2, nand_or = evaluate(a, b, c)
    print(
        f"{a} {b} {c} | {xor}      {nand}      {nor}"
        f" |      {f1} / {f2}      |      {nand_or}"
    )
```

Run it directly if you saved it:

```
python3 boolean_recipes.py
```

Or paste it into:

```
python3
```

Predictions—write these first

1. On which (A, B) rows will the derived XOR column be 1?
2. Will the NAND and NOR columns ever both be 1? If so, on which row?
3. Earlier you predicted how many rows make $F = \neg(A \vee B) \vee C$ true. What is your count?

4. Will `F original` and `F rewrite` differ on any row?
5. Does `OR-via-NAND` depend on `C`?
6. For every row, should `OR-via-NAND` equal `A OR B`?

Observe

Run the program. If a prediction was wrong, do not merely replace it with the observed value. Identify the faulty step:

```
I treated OR as exclusive.  
I forgot that C = 1 makes the final OR true.  
I moved NOT through OR but forgot to change OR to AND.  
I read 0 as inactive even though the signal was active-low.
```

Extend

Add two columns:

```
AND-via-NOR  
XOR-direct
```

Use only Python's `not` and `or` operations to construct the first one in the same shape as the NOR derivation. Use `a ^ b` for the second. Confirm:

```
AND-via-NOR == (A AND B)  
XOR-direct == XOR-derived
```

The program is a simulator. It calculates the same input/output mapping as the intended gates, but the CPU running Python is already a vastly larger machine built from physical logic. Software evaluation is evidence about the truth table, not a transistor-level implementation of the little circuit you drew.

9. Checkpoint

Close the lesson and answer from memory.

Explain

1. Derive XOR as an OR of two mutually exclusive AND cases.
2. Explain NAND and NOR in words and by expression.
3. State both De Morgan laws and describe what changes when NOT crosses a gate boundary.
4. Explain why an active-low reset is asserted at `0`.
5. Define functional completeness without saying “because NAND is universal.”
6. Explain how one mismatching truth-table row disproves equivalence.

Draw

On paper, draw:

1. $Y = (A \vee B) \wedge \neg C$;
2. NOT using one NAND;

3. AND using only NANDs;
4. OR using only NANDs;
5. NOT using one NOR;
6. OR using only NORs;
7. AND using only NORs.

Label every intermediate wire. Then trace one input row through each network.

Build a table

Without code, derive all eight rows for:

$$Y = \neg(A \wedge B) \wedge (B \vee C)$$

Use these intermediate columns:

$$\begin{aligned} P &= A \wedge B \\ Q &= \neg P \\ R &= B \vee C \\ Y &= Q \wedge R \end{aligned}$$

Then answer:

- Which input rows make $Y = 1$?
- What does the expression mean in one plain-language sentence?
- Can you rewrite Q with De Morgan's law?
- Does the rewrite produce the same final column on all eight rows?

Mastery prompt

How can NAND alone describe every Boolean function, yet two NAND-only implementations of the same function still differ as pieces of hardware?

A strong answer connects:

- construction of NOT, AND, and OR from NAND;
- truth tables as complete stable-behavior specifications;
- Boolean equivalence versus physical identity;
- logic depth, delay, area, power, loading, and available technology.

If you can rebuild that explanation rather than recite it, you have the mental model needed for Day 10.

Mental model

```
verbal condition
  ↓ select desired input cases
Boolean expression
  ↓ expose intermediate operations
circuit recipe
  ↓ enumerate all input combinations
truth table
  ↓ compare output columns
equivalence proof
  ↓ choose among equivalent forms
physical implementation tradeoffs
  ↓ compose at larger scale
CPU logic blocks
```

The sentence to keep:

A truth table tells us whether two circuits compute the same Boolean function; it does not tell us that they cost the same or behave identically during physical transitions.

Why? notebook

1. Why is XOR not just another spelling of OR?
2. Why do intermediate truth-table columns reduce errors?
3. Why does matching every row establish equivalence for a fixed set of Boolean inputs?
4. Why must both the operator and the input polarities change in De Morgan's transformation?
5. Why can LOW mean “asserted” without contradicting binary logic?
6. Why does tying a NAND's two inputs together create NOT?
7. Why does constructing NOT, AND, and OR establish NAND's functional completeness?
8. Why does universal not mean physically optimal?
9. Why might a CPU designer accept more gates in exchange for less logic depth?
10. Why can a truth table hide glitches and propagation delay?
11. Where does the meaning of a signal such as `RESET_n` come from: voltage, gate topology, naming convention, interface specification, or some combination?
12. Why can two circuits be logically equivalent while drawing different power or occupying different area?

References

Local

- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems: From Bits and Gates to C and Beyond*, 2nd ed., Chapter 3, §§3.1–3.3 and §3.3.5, book pages 51–64: transistor switches, basic gates, De Morgan's laws, combinational logic, and logical completeness.

Local file: `source-materials/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf`

- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 10–12, local PDF pages 98–145: switches, logic gates, relay networks, and the bridge toward binary computation. Local file: `source-materials/library/books/01-foundations-and-architecture/code-hidden-language-computer-hardware-software-charles-petzold.pdf`

Web

- MIT OpenCourseWare, *Computation Structures*, “Boolean Algebra” notes, including truth tables, Boolean identities, De Morgan's laws, and sum-of-products construction: <https://computationstructures.org/notes/cmos/notes.html>
- Nand2Tetris, Project 1, “Boolean Logic,” gate interfaces and construction from primitive NAND: <https://www.nand2tetris.org/project01>
- All About Circuits, “Active-Low and Active-High Logic,” signal assertion and bubble notation: <https://www.allaboutcircuits.com/textbook/digital/chpt-3/active-low-and-active-high-logic/>
- MIT Libraries, Claude E. Shannon, *A Symbolic Analysis of Relay and Switching Circuits* (1937 thesis), the foundational connection between Boolean algebra and switching networks: <https://dspace.mit.edu/handle/1721.1/11173>

Next bridge: use expressions and truth tables to design small combinational blocks deliberately, beginning with one-bit arithmetic and then reasoning about how wider CPU datapaths are assembled.